Figure 7.2 A web consumer making a single query to a GraphQL endpoint to retrieve all the data they require

GraphQL defines the data they expose through their own type language which is called the GraphQL schema definition language (SDL). This schema can be seen as the contract that the consumers need to abide by when making queries. An example schema for our movie is shown here:

```
type Movie {
  id: Int
  name: String
  year: Int
}
```

As we first mentioned in chapter 3, a schema provides the outline or the structure of a JSON file as an example. The `type Movie` doesn't expose any functionality yet, but it only defines the structure of a movie. In GraphQL, the behavior of the API is defined separately and is done by a resolver function. Resolvers are responsible for knowing which data to fetch. An example resolver for getting a single movie is shown here:

```
const resolvers = {
  Query: {
    movie: (root, { movieId }, { dataSources }) => {
      return dataSources.movieApi.getMovieById(movieId);
    }
  }
}
```

With the schema and resolvers defined, if a consumer adds "rating" as part of their query, but the GraphQL SDL doesn't include "rating," then the query will fail.

Because the structure of a query can be defined with a schema, should you still do contract testing? Yes, you should! With contract testing, teams maintaining the GraphQL API can understand the following:

- How different consumers are using the API
- What query from consumers produces bad output
- What query from consumers produces different status codes, such as 401 or 403